# RCM & Billing Decision Guide

Use this guide to decide which parts of the revenue cycle your application owns and which it delegates to billing partners. Work through coverage, charge capture, claims, remittance, and patient responsibility to define the data model, integrations, and operational handoffs for your reimbursement model.

*Companion to the Billing docs.*

Start with reimbursement and ownership, then follow the relevant financial workflows:

1. **Choose your reimbursement model and workflow ownership.** Decide what your application owns and what a billing partner or existing system handles.
2. **Define coverage and accounts and checks before service.** Establish responsibilities for balances, eligibility, authorization, and estimates where applicable.
3. **Define charge capture and claim submission.** Decide how services are coded and priced; include claims when billing payers or when a partner requires that submission format.
4. **Assign payer-response and accounts-receivable responsibilities.** Define posting, denials, follow-up, and reporting, including the boundary with any partner that owns them.
5. **Define patient billing** if patients owe balances or pay directly. Decide who collects payments and provides statements or superbills.
6. **Plan value-based contracts** if you operate under capitation or risk-based arrangements.

Use the questions below to establish your use case, then follow the relevant sections. Add requirements beyond these examples where needed.

## Before you start

### What is your reimbursement model?

- Insurance / claims-based – you bill payers and reconcile their responses
- Patient self-pay / cash – patients pay you directly; no payer claim
- Value-based / capitation – per-member-per-month or risk-based
- Some combination

*Why: Claims-based workflows cover every applicable decision; direct self-pay uses charge capture and patient billing. Partner-managed self-pay may also require claim assembly as the partner's submission format (Patient billing). Value-based workflows start with the coverage and account model, then the value-based decision (Value-based contracts).*

### How much of the revenue cycle runs in Medplum versus delegated?

- **Clearinghouse-direct** – Medplum runs the cycle; the clearinghouse is transport. You own scrubbing, denials, and remittance posting. The most complete documented path.
- **RCM-automation partner** – you send encounter and charge data and delegate the back end; agree which scrubbing, payer routing, denials, resubmission, patient collections, and reporting workflows the partner owns. Define the submission interface and which billing tasks or status changes sync back into Medplum.
- **Documents only (no electronic claims)** – you don't run claims through a clearinghouse or RCM partner. Generate a CMS-1500 PDF for the occasional paper/portal payer, or hand patients a superbill to self-submit for out-of-network reimbursement. Common for cash-pay and OON practices (see Reimbursement model); also a per-payer fallback you can layer onto any engine above.

- **External Practice Management (PM) / billing system** – Medplum is the clinical source of truth; charges or claims hand off to an existing PM/billing system that owns the rest.

*In one line: a clearinghouse moves your claims; an RCM platform runs your revenue cycle. Either way Medplum owns the front half (coverage, charge capture, claim assembly); the paths diverge on the back half.*

| Revenue-cycle stage | Clearinghouse | RCM partner | Documents only | External PM |
|---|---|---|---|---|
| Coverage & charge capture | You build | You build | You build | You build |
| Eligibility | Eligibility integration; you own timing and follow-up | Partner workflow or separate eligibility integration | Manual or separate eligibility integration | External |
| Claim assembly | You build | You build | You build | External |
| Scrubbing / validation | You build | Partner owns | Light | External |
| Submission | You build (837P) | Partner submit op | PDF / patient files | External |
| Responses & ERA posting | You own report ingestion, interpretation, and posting; verify what the integration supplies | Partner owns; define status/task sync | Manual | External |

| Revenue-cycle stage | Clearinghouse | RCM partner | Documents only | External PM |
|---|---|---|---|---|
| Denials & resubmission | You build | Partner owns; define status/task sync | Manual | External |
| AR & reporting | You build | Partner owns | Manual | External |

**How to choose:**

| | Clearinghouse | RCM partner |
|---|---|---|
| You get | Full control; billing data stays as FHIR in Medplum | A payer-rules engine and in-house billers; far less to build |
| Trade-off | You build and maintain the entire back office | Recurring fees; less control; AR and reporting live on the partner's platform |
| Choose if | You have RCM expertise and want a fully owned cycle | You want to outsource the cycle |

*Paths can mix. The choice sets which workflows you build (Charge capture, Claim submission, Responses and posting, Denials and appeals, Patient billing, AR and reporting) vs. read only for the boundary.*

## Who does the billing work, and is the patient part of it?

- Charge capture: clinicians at point of care, or back-office entry?
- Coding and claim work: dedicated coders/billers, or automated with exception review?
- Accounts receivable (AR) follow-up: who works denials, aging, and patient balances?
- Patient: do patients see estimates, statements, or pay online?

*Why: drives charge-capture design (Charge capture), who works denials and AR (Denials and appeals, AR and reporting), and whether patient-facing billing is in scope (Patient billing).*

### Where does coding happen, and how automated is it?

- Auto-derived from clinical data (visit type, orders, procedures)?
- Coder-reviewed before the claim is built?
- Fully manual?

*Why: drives the coding and review model in charge capture (Charge capture) and where scrubbing exceptions surface (Claim submission).*

### New build or replacing/extending, and what else must billing touch?

- If replacing: what billing/PM system, and what gaps drove the change?
- If new: is billing core to the product (you're building an RCM/billing product) or additive (a clinical app that also needs to bill)?
- What other systems must billing read from or write to (clinical/EHR source, PM, clearinghouse, accounting/GL, payment processor, payer portals)?

*Why: surfaces integration boundaries and the source-of-truth split behind Workflow ownership, Claim submission, and Responses and posting. Core-to-product builds tend to own the cycle (a clearinghouse); additive builds tend to delegate (an RCM partner or external PM).*

## Coverage and accounts

### How will coverage, subscribers, and payers be represented?

How you represent a patient's insurance (the coverage stack, the subscriber, payers, cost-sharing) is the foundation every claim and eligibility check builds on. See Representing Patient Insurance Coverage; coverage capture at intake is in the Intake guide.

**Questions:**

- Do patients commonly have more than one coverage, and must you bill them in order?
- Is the subscriber sometimes someone other than the patient?
- Is your payer list curated in Medplum, synced, or created on demand?
- What cost-sharing (copay, coinsurance, deductible) do you need for estimates and patient billing?

| Situation | Approach |
| --- | --- |
| Subscriber is the patient | One Coverage; subscriber = Patient; relationship = self; payor ⟶ Organization. |
| Primary + secondary | One Coverage per plan; Coverage.order sets billing sequence. Secondary submits after the primary remittance posts (Responses and posting). |
| Subscriber ≠ patient | Coverage + RelatedPerson for the subscriber. The relationship inverts: Coverage.relationship is the patient's relationship to the subscriber, the RelatedPerson's is the reverse (coverage child ⟶ related person parent). Getting this backwards is a common billing bug. |
| Member ID & plan details | Coverage.subscriberId for member ID; Coverage.class for group, plan name, RxBIN/RxPCN. |
| Cost-sharing | Coverage.costToBeneficiary; feeds estimates (Authorization and estimates) and patient responsibility (Patient billing). |
| Payer directory | Medplum ships a built-in **Payer Directory** of Organizations, or curate your own. Verify the identifier system for each eligibility and claim-submission route; they may differ even for the same partner. |
| Card images | Store as DocumentReference/Binary linked to the Coverage. |

## Does the application need an account and financial ledger?

Decide whether you track a patient financial account that rolls charges, claims, and payments into a balance (the backbone for AR in AR and reporting and patient billing in Patient billing), or let each charge stand alone.

FHIR R4 supplies account and billing resources, but an Account is not a computed ledger. Choose the posting model, claim-to-account association, reversals, and reconciliation rules explicitly; these are application responsibilities.

**Questions:**

- Do you need a single balance rolling up charges, payer payments, adjustments, and patient payments?
- Is the account scoped per patient, per visit, or per episode?
- Do you track payer and patient responsibility separately on the same account?
- Must the account reconcile against an external accounting/GL system?

| Situation | Approach |
|---|---|
| Account links | Use ChargeItem.account, Invoice.account, and Encounter.account to reference an Account. R4 Claim and PaymentReconciliation have no dedicated account element |
| Account scope | Set Account.subject to the Patient. For a visit account, link it from Encounter.account; Account.subject does not reference Encounter |
| Claims and remittances | Define an explicit claim-to-account association in your application model. ClaimResponse.request links to Claim; PaymentReconciliation.detail.request and .response can identify the related request and response. These links do not post money |

| Situation | Approach |
| --- | --- |
| Running balance | Calculate charges, payments, adjustments, and reversals from uniquely identified postings. R4 Account has no `balance` field; decide where the postings and calculated totals live |
| Payer vs. patient | Track responsibility allocations without counting the same charge again when creating a patient Invoice. Decide whether Medplum or the partner owns each balance |
| No ledger | Skip `Account` for pure pass-through submission where another system owns the balance, but you lose in-Medplum AR (AR and reporting). |

For example, a visit can have `Account.subject` → `Patient`, `Encounter.account` → `Account`, and `ChargeItem.account` → `Account`. A patient Invoice can use `Invoice.account` → `Account` and `Invoice.lineItem.chargeItemReference` → `ChargeItem`. Associate the submitted Claim with that account separately, and record each applied remittance/payment once. The resource links support reconciliation; creating these resources alone does not calculate a balance.

## Checks before service

### What eligibility and benefit evidence is required, and when?

Decide whether you verify coverage before service, what you ask the payer, and when the check runs. See Insurance Eligibility Checks.

Choose whether eligibility runs through your clearinghouse, RCM partner, or a separate integration, including whether real-time X12 270/271 exchange is needed. Confirm supported service types, payer routing identifiers, and how responses map to CoverageEligibilityResponse resources. See Related Integrations for the available first-party routes.

**Questions:**

- What must you confirm: active coverage, covered service, in-network status, or benefit/cost detail?
- When does the check run: scheduling, check-in, or batch pre-visit?
- Do you re-check on a cadence for ongoing patients?
- Who works failed or ambiguous checks?

| Situation | Approach |
|---|---|
| Verify coverage / benefits | Create a CoverageEligibilityRequest with purpose: benefits and invoke the selected eligibility operation; retain the response and source evidence |
| Service-specific benefits | Request supported X12 service types through CoverageEligibilityRequest.item.category. A payer may still return general or incomplete benefits; define the fallback when the answer is insufficient |
| In-network check | Interpret the benefit-network indicator in context. It is not independent verification of the provider's contract, location, or service; ambiguous cases need payer confirmation |
| Cost-sharing | Copay, coinsurance, remaining deductible, and out-of-pocket land in CoverageEligibilityResponse.insurance.item; the input to patient estimates (Authorization and estimates). |
| When it runs | A Bot issues the request on the triggering event; persist the response for the visit and billing. |
| Exception | On failure or ambiguity, create a Task for registration rather than proceeding silently. |

CoverageEligibilityResponse.outcome = complete means processing completed without reported errors. It does not establish coverage for every service, network participation, or prior-authorization approval. Use the evidence and exception-handling approach in the Referrals Decision Guide.

## How will prior authorization and patient estimates be handled?

Decide whether services need payer authorization before they happen, how you track that to a decision, and whether you produce patient cost estimates up front.

*Note: the eligibility- and Claim-based rows below are one FHIR approach to prior auth; validate against your payer requirements. The standards-based CMS-0057-F path (CDS Hooks / CRD) is in alpha. Prior auth in a referral context is in the Referrals guide.*

**Questions:**

- Do any services require prior authorization before scheduling or delivery?
- How do you check whether auth is required, and track a submitted auth to approval/denial?
- Must scheduling be gated until authorization clears?
- Do you owe patients a cost estimate before service (e.g. a Good-Faith Estimate)?

| Situation | Approach |
|---|---|
| Is auth required? | Collect requirements for the service, payer, provider, and date. Use `purpose: auth-requirements` only through a route that supports it; a benefits check alone may not answer the question |
| Submit a prior auth | A FHIR model can use `Claim.use: preauthorization` and a corresponding ClaimResponse, but you must select a supported payer/vendor submission workflow. Professional-claim submission does not imply prior-auth transport support |

| Situation | Approach |
| --- | --- |
| Standards-based (CMS-0057-F) | Electronic prior auth via CDS Hooks / Coverage Requirements Discovery (CRD); alpha. The emerging payer-interop path as the rule phases in. |
| Gate scheduling | Hold work in an auth-pending `Task.businessStatus` until the required approval evidence is verified for the service, dates, and units. Processing outcome `complete` alone is not approval |

**Good-Faith Estimates (No Surprises Act).** One FHIR approach: build the estimate from pricing rules (`ChargeItemDefinition`/`$apply`, Charge capture) and the patient's cost-sharing (`Coverage.costToBeneficiary`, Coverage), delivered as a `DocumentReference`; validate against your compliance requirements. Mainly for self-pay and out-of-network patients.

## Charges and claims

### What creates a charge, and how is it coded and priced?

*For insurance claims, cover every subsection. Direct self-pay needs pricing (Charge capture), patient billing (Patient billing), and AR ownership (AR and reporting); partner-managed self-pay may also need claim assembly and submission (Claim submission, Patient billing).*

Decide what gets billed, where the codes come from, and how each line is priced. Coding is the largest source of denials, so treat it as first-class, not a byproduct of charge entry.

**Questions:**

- What generates a charge: a completed encounter, an order/procedure, a fulfilled service?
- Where do codes come from (CPT/HCPCS, ICD-10, modifiers): auto-derived, coder-reviewed, or manual (Coding ownership)?
- Is pricing a single fee schedule, or does it vary by payer contract?

- Who reviews coding before a claim is built, and how do exceptions surface?

| Situation | Approach |
|---|---|
| What gets billed | One ChargeItem per billable line; ChargeItem.code carries CPT/HCPCS. Link context → Encounter and reference the Account (Accounts) at creation, or reconciling into claims and AR later gets painful. |
| Pricing | ChargeItemDefinition holds the rules; $apply calculates the price. |
| Payer-specific rates | Multiple ChargeItemDefinitions keyed to payer/contract; apply the one matching the coverage. |
| Coding source | Auto-derive via a Bot, or set by a coder; route low-confidence items to a review Task. |
| Diagnoses | Capture ICD-10 diagnoses to attach to the claim (Claim.diagnosis, with item-level pointers) in Claim submission. |

## How will claims be assembled, validated, and submitted?

Decide how you build a valid claim from charges and coverage, and how it leaves your system. The workflow ownership choice in Workflow ownership plays out here.

⚠ **One-way door:** Choosing a submission backend is hard to reverse. Each maps the Claim to a different format (837P, the partner's API, a CMS-1500 PDF) with a different response pipeline (Responses and posting), and payer identifier systems differ. Keep one Claim as the source of truth so the backend stays swappable.

**Questions:**

- Are you billing as an organization or an individual provider?

- Who rendered the service, and which service facility should appear on the submission?
- Which submission path(s), and does it vary by payer (electronic for most, paper/portal for some)?
- What validation must pass before submission, and where do failures surface?
- Do you submit secondary claims after the primary pays (coordination of benefits)?

| Situation | Approach |
| --- | --- |
| Build the claim | Claim referencing the Coverage (Claim.insurance), charges (Claim.item), and ICD-10 diagnoses (Claim.diagnosis) with item-level pointers. |
| Billing provider | Set on Claim.provider: the billing Organization (or a Practitioner for individual billing). The rendering provider goes on Claim.careTeam with role primary. |
| Service facility | Set Claim.facility to the service Location when required. Confirm the submission route's facility requirements and fallback behavior, including individual billing and telehealth |
| Electronic (you own the cycle) | The clearinghouse submit operation sends an 837P and writes a correlation id back to the Claim. |

| Situation | Approach |
| --- | --- |
| RCM partner | Submit through the partner's supported interface; persist its acknowledgment and correlation identifiers. Agree which downstream workflows it owns and how billing tasks or status changes sync back (Denials and appeals). |
| Manual / paper / portal | `Claim/$export` generates a CMS-1500 PDF (`Binary`; experimental); log the outbound document. |
| Patient-submitted (out-of-network) | Generate a superbill for the patient to file. |
| Validation and submission failure | Validate before submit and surface failures in a Task. Use the returned ClaimResponse outcome/errors when available and handle thrown invocation errors separately. `error` is not a valid R4 Claim.status; keep submission state distinct from the Claim lifecycle |
| Multiple channels | One `Claim` is the source of truth; run separate send paths per payer without diverging content. |

## Payer responses and accounts receivable

### How will payer responses be matched and remittances posted?

Decide how you ingest what the payer sends back (the *277CA* acknowledgment, accepted or rejected pre-adjudication, and the *835 ERA* remittance, what was paid, adjusted, or denied line by line) and how payments post. These share one pipeline, so design them together.

*On the clearinghouse path, distinguish report ingestion from adjudication interpretation and posting. Verify which steps the integration implements and build the rest. With an RCM partner, confirm its posting responsibilities and the status or task updates available to your application.*

**Questions:**

- How do responses arrive: webhook push, polling, or partner API?
- How do you match a response to the originating claim?
- How do payer payments and adjustments post to the account?
- What share posts automatically vs. needs review?

| Situation | Approach |
|---|---|
| Ingestion | Define webhook delivery, polling, and recovery of missed reports. Deduplicate by the source transaction identifier; make interpretation and posting independently idempotent |
| Acknowledgment (277CA) | Retain the raw acknowledgment, for example as a DocumentReference, and correlate it with the originating submission. Distinguish acceptance or rejection from adjudication |
| Remittance (835 ERA) | Retain source reports and map payment, adjustment, and responsibility details into your posting model. Storing a report alone does not create PaymentReconciliation resources or post money |
| Matching | Use the submission's patient-control-number and correlation identifiers. Provide an unmatched-report queue rather than assuming every report can be linked automatically |

| Situation | Approach |
| --- | --- |
| Source claim status | Define where external status is stored and how it advances your workflow. Keep submission outcome, payer status, adjudication amounts, and payment posting distinct |
| Adjudication and posting | Build the report-to-ledger mapping, including line amounts, responsibility, adjustments, and reversals. If using PaymentReconciliation or additional ClaimResponses, define their role and correlation explicitly |
| Review | Auto-post only after the application's validation and reconciliation rules pass; route mismatches and ambiguous reports to a Task |

## Who works rejections, denials, resubmissions, and appeals?

Decide how you work claims the payer rejects or denies: correcting and resubmitting, or appealing.

*Note: The rows below are one FHIR approach to denial handling; validate against your workflow. On the clearinghouse path you build this in Medplum; with an RCM partner, confirm how its denial work and status changes surface back in your application.*

**Questions:**

- Do you work front-end rejections (277CA) differently from adjudicated denials (835)?
- When you fix and resend, is it a corrected claim or a fresh submission?
- Do you track appeals (deadlines, supporting documentation, outcome)?
- How do staff find claims that need work?

| Situation | Approach |
| --- | --- |
| Surface denials | Interpret rejection or denial detail from the submission response and stored payer reports (Responses and posting); create a Task work queue for the biller |
| Corrected claim | Resubmit a corrected Claim linked to the original (Claim.related), via the same backend (Claim submission). |
| Retry a rejected submission | Check the route's retry and duplicate-submission behavior. Preserve prior responses and submission history; do not delete active ClaimResponses simply to bypass a guard. An accepted submission needs a deliberate correction workflow |
| Appeals | Track as a Task with deadlines; attach supporting docs as DocumentReference. |
| Find work | Search over open denial Tasks (AR and reporting). |
| Partner-owned | Agree who works denials and resubmission, which exceptions return to your team, and how status or task updates enter Medplum. |

## How will staff track balances, aging, and unresolved work?

Decide how you track where every claim and balance sits, and how staff find work that's stuck or overdue.

*Note: The rows below are one FHIR approach to AR tracking; validate against your reporting needs. Most relevant on the clearinghouse path; an RCM partner owns AR and reporting.*

**Questions:**

- What lifecycle states matter (e.g. draft → submitted → accepted → paid → denied → closed)?
- How do you standardize states so queues, reports, and integrations agree?
- Do you track aging (days since submission, days pending) and AR buckets?
- How do staff find overdue or stuck claims and balances?

| Situation | Approach |
| --- | --- |
| Lifecycle states | Track progress with `Task.businessStatus`; move `Task.status` on clear transitions. Keep the state set small and single-sourced. |
| External signals | Bots interpret submission responses and payer reports, then advance the appropriate workflow stage. Keep accepted, adjudicated, and posted states distinct |
| Aging & AR | Record submission and stage-entry times, derive stage intervals from history or explicit transition records, and age calculated outstanding balances. `Claim.created` is not submission time; `Task.lastModified` may reflect unrelated edits |
| Overdue work | Scheduled `Bot`s or subscriptions flag items past an SLA into work queues. |

## How will patients be billed?

Decide how you bill patients for their share (copays, coinsurance, deductibles, full self-pay) and how they pay.

**Questions:**

- What balances do you bill: post-adjudication responsibility, self-pay, or both?
- Will you collect self-pay directly or delegate it to a billing partner?
- Do you issue statements, and on what cadence?

- Do you take online payment / card on file, through which processor?
- For out-of-network patients, do you provide a superbill instead of billing the payer?

| Situation | Approach |
|---|---|
| Patient balance | Invoice for patient-owed amounts rolling to the Account (Accounts); responsibility from remittance (Responses and posting) or self-pay pricing (Charge capture). |
| Online payment | Integrate a payment processor; a sample integration shows keeping invoices and payments in sync. |
| Direct self-pay / cash | Price via ChargeItemDefinition/$apply (Charge capture) and invoice the patient directly; no payer claim |
| Partner-managed self-pay | Confirm the partner's encounter/charge submission format and how it represents self-pay responsibility. If that format uses a FHIR Claim, distinguish this data handoff from an insurance claim sent to a payer |
| Out-of-network | Provide a superbill (Claim submission) for the patient to submit for reimbursement. |

## How will value-based contracts work?

Value-based care isn't a claims pipeline; it splits into measurement (quality, gaps in care) and money (capitation, risk, settlement). Read only for capitated or risk-based contracts.

*Note: Quality measurement uses documented operations; the financial rows are one FHIR approach to validate against your contracts. Quality/HEDIS reporting overlaps clinical analytics; see Analytics.*

**Questions:**

- Which contracts are capitated or risk-based, and which members are attributed to you?
- Which quality measures (eCQM, HEDIS, stars) are you accountable for?
- Do you still submit claims or encounter data when capitated (for risk adjustment and measurement)?
- How are capitation payments and any shared-savings settlement reconciled?

| Situation | Approach |
| --- | --- |
| Attribution / panel | Group as the member roster; ingest the payer roster via a Bot; CareTeam for accountable teams. |
| Quality measurement & gaps in care | Measure + MeasureReport via Measure/$evaluate-measure; eCQM and HEDIS via Analytics. |
| Risk adjustment (HCC / RAF) | Capture Condition coding from encounters and submit as encounter data; ties to coding (Charge capture). |
| Encounter data | Even when capitated, reuse claim assembly (Claim submission) to send 837/encounter data for measurement and risk. |
| Payments (PMPM, settlement) | If using PaymentReconciliation, define its association to contract, period, and account in the posting model (Accounts). Receiving a reconciliation resource does not automatically change an Account balance |

## Related Integrations

See the integration docs for setup and implementation details:

- Stedi: clearinghouse connectivity for eligibility, professional claims, and claim responses.

- Candid Health: RCM partner integration for claim submission and billing-task synchronization.